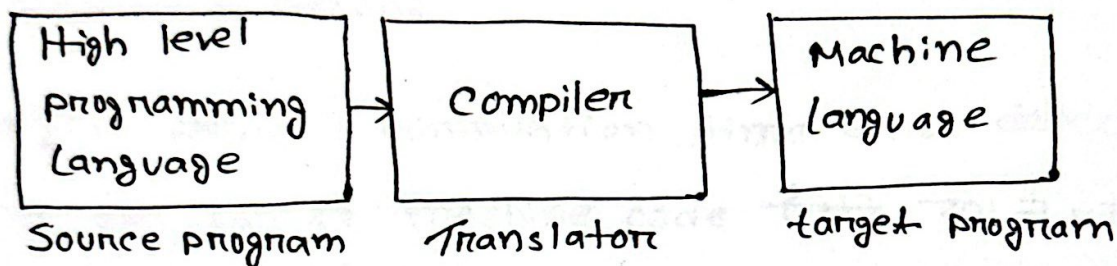


Compiler Design

What is a compiler?

- High level programming language ko translator translate করে machine language এ convert করে, যা computer বুঝতে পারে। এখানে translator শিখার যা কাজ করে, তাই compiler.



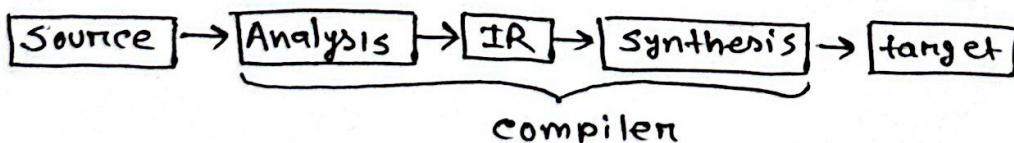
Structure of a compiler

২ phase of compiler

↳ Analysis phase

↳ Synthesis phase

Analysis phase এর কাজ source program ko ডালাডারে বুঝতে পারা, এরপর তাকে intermediate representation তৈরি করে। Intermediate representation থেকে Synthesis phase convert করে target program (machine language)।



→ compiler's frontend

Analysis phase: Source code ko jankhane ke liye dekha jata hai, jahan IR (Intermediate Representation) banaya jata hai. IR (Intermediate representation) source code ke ek aise form hai, jo machine ke liye samajhne mein aasaan hai. compilation time error detect karne ka ek phase.

Synthesis phase: IR ko aise phase machine language mein convert karke deta hai, jahan compilation time error detect karke deta hai.

→ compiler's backend

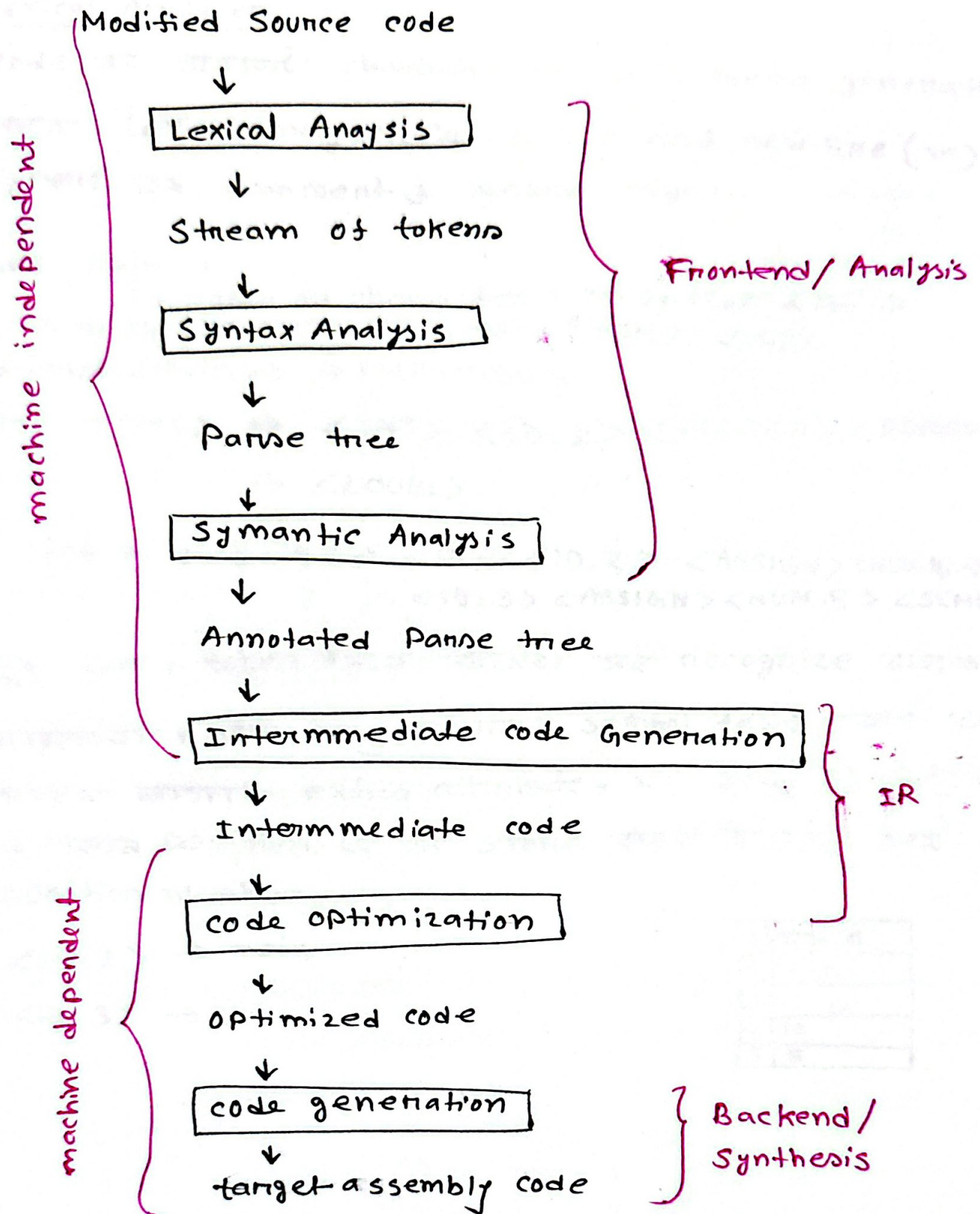
Analysis phase aur compilation time error detect karne ke liye aise form machine code mein convert karke deta hai. aur synthesis phase ke liye deta hai.

Nitrin[®] SR
Nitroglycerin USP 2.5 mg & 6.4 mg Tablet


Healthcare

VESTAR MR[®]
TRIMETAZIDINE

Compilation



Lecture 2

Lexical Analyzer

code এর প্রত্যেকটি character পাড় এবং token generate করে। Lexical analyzer এ space and new line (\n) ignore হয়, comment ও ignore হয়।

```
int main()
```

↳ name on character combination <ID, >
↳ recognize করে C keyword হিসেবে <INT>

```
#include <stdio.h> → preprocessing
```

```
int main() → <INT> <ID, 1> <LPAREN> <RPAREN>  
{ → <LCURL>
```

```
int a=10, b=5; → <INT> <ID, 2> <ASSIGN> <NUM, 10> <COMMA>  
               <ID, 3> <ASSIGN> <NUM, 5> <SEMI>
```

খুব <ID> token দিয়ে আমরা সব recognize করতে পারবেনা। এর জন্য প্রয়োজন symbol table. তাই ID এর সাথে লাগবে extra attribute, যা হচ্ছে symbol table এ কোন position এ তা store করা হয়েছে সেই position number.

<ID, 1> → main

<ID, 3> → b

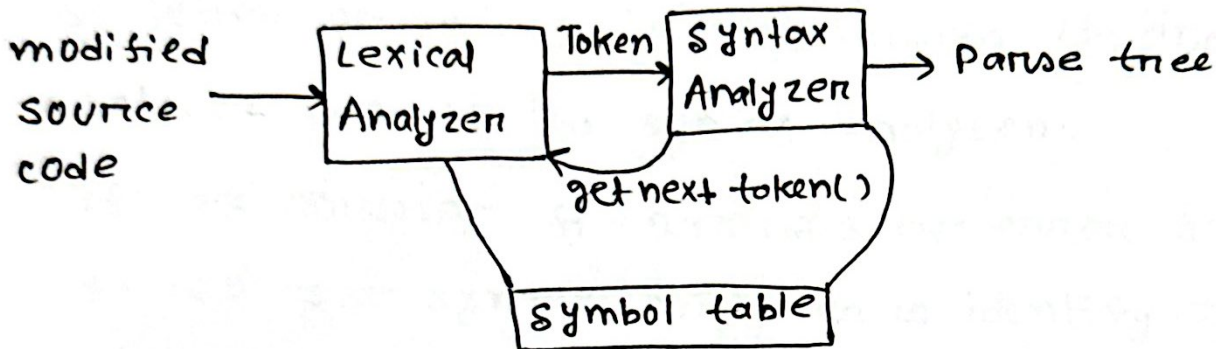
1	main
2	a
3	b
4	10
5	5

The role of a lexical Analyzer

- source code এর প্রতিটি character পর

- comment, space, tab, newline

doesn't generate any token for these, just recognize them.



ID = letter followed by letter or digits

✓ int abc;

lexeme → abc

✓ int a123;

ID → token

✗ int 123a;

pattern → যাৰ সৈতে match কৰা (খ),
আৰু ID বুলি

Example of tokens

Token	Informal description	sample lexemes
if	characters i, f	if
else	characters e, l, s, e	else
comparison	< or > or <= or >= or == or !=	<=, !=
id	letter followed by letters and digits	pi, score, D2
number	any numeric constant	3.1416, 0, 6.02
literal	anything but ", surrounded by " 's	"core dumped"

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet


Healthcare

VESTAR MR[®]
TRIMETAZIDINE

lexical error

$\{ (a > b) \}$
...
 $\}$

$\langle id, \rangle$

↳ letter followed by
letter or digit.

lexical analyzer check karta id, or letter followed by letter or id. So 'si' maintains it and lexical analyzer pass it to syntax analyzer.

if error aagaya si maintain kr error dene parhata hai
so isse hum syntax analyzer ko identify krte.

Regular Expression

a^* → klen closure
 $a^* \rightarrow \epsilon, a, aa, aaaa, \dots$

a^+ → positive closure
 $a^+ \rightarrow a, aaa, aaaa, \dots$

$$ID = [A-Za-z][A-Za-z0-9]^*$$

$= CAIZ$

Regular definition

letter = $[A-Za-z]$, digit = $[0-9]$

$ID = [A-Za-z][A-Za-z0-9]^*$

$\Rightarrow ID = \text{letter}(\text{letter} | \text{digit})^*$

digit $\rightarrow [0-9]$

digits $\rightarrow \text{digit}^+$

num $\rightarrow \text{digits} (\backslash \cdot \text{digits}) ? (E [+ -] ? \text{digits}) ?$

() ? $\rightarrow$ optional

Action

{ ID }

{ store SymbolTable (); }

return ID;

" int "

{ return INT }

delim $\rightarrow [\backslash n \backslash t]$

{ delim }

{ }

\n

{ line++; }

*

INT $\rightarrow$ " int "

FLOAT $\rightarrow$ " float "

ID $\rightarrow$ letter (letter | digit)*

lexical analyzer is order matter করে। এখানে যদি

ID লিখা হাত INT, FLOAT এর আগে, অর্থাৎ

'int a; $\rightarrow$ এই ক্ষেত্রে 'int' ID এবং INT দুইটাই indicate করে, কিন্তু 1st token আগে থাকবে, তা return করে। ID আগে থাকলে তা return করে, তা

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet



VESTAR MR[®]
TRIMETAZIDINE

হুন্। প্রকৃতই আমাদের INT return করা উচিত।
আই আমাদের INT, FLOAT এবং ID এর আগে লিখতে।

* int inta;

↳ <INT> <ID> ✓

<INT> <INT><ID> ✗

Lexical Analyzer সবার প্রথম longest matching করার
try করে। যদি tie হয়, তাহলে যার order আগে
আছে আগে নিতে।

* PP → "++"

PLUS → "+"

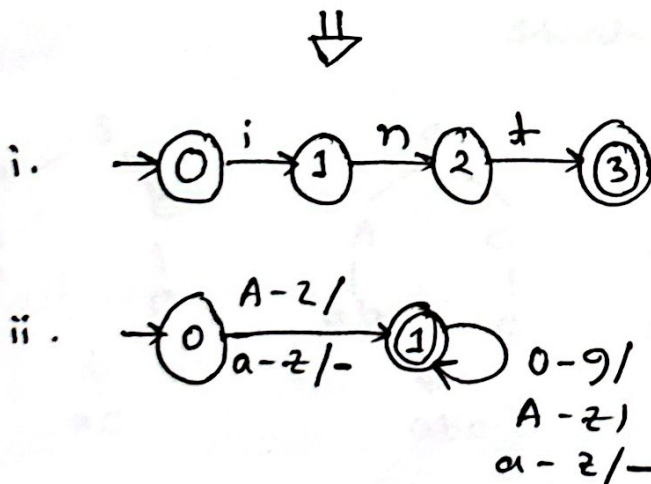
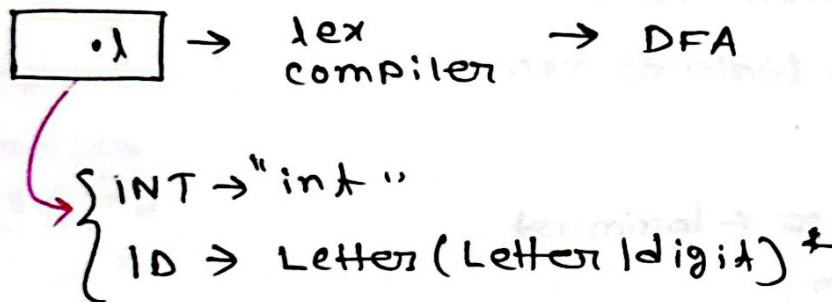
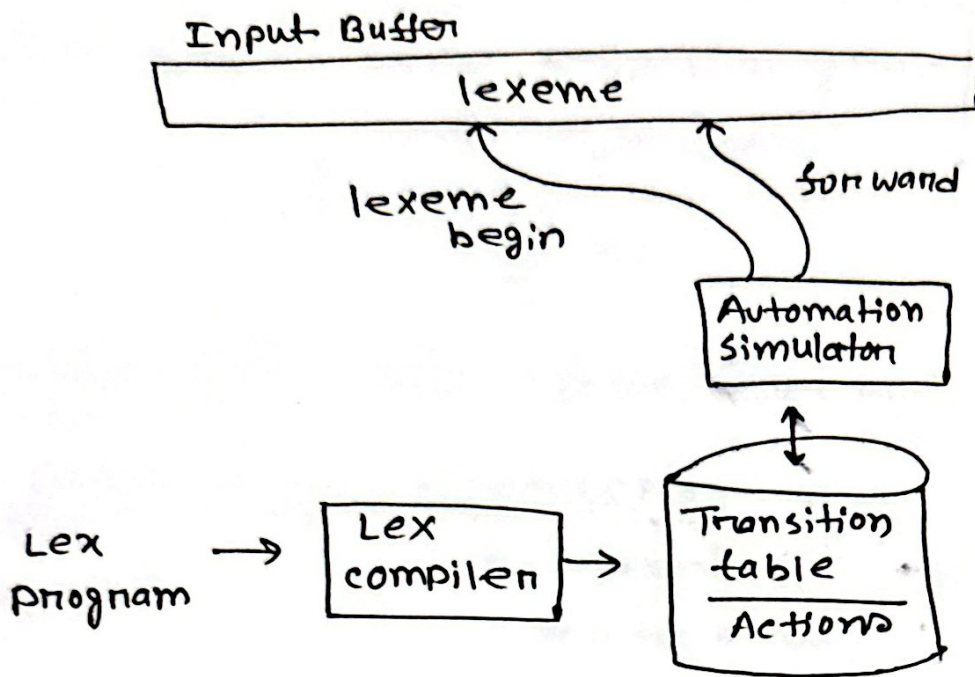
a++;

<ID> <PP> <SEMI> ✓

<ID> <PLUS> <PLUS> <SEMI> ✗

} longest matching
preferable

Generation and structure of a Lexical Analyzer



Lex program is rule, regular expression, action लिखि। Or lex compiler लिखि। Or regular expression or corresponding DFA बनाय। DFA को input से आरम्भ करके simulate करते हुए pointer दिखाते हैं। lexeme begin and forward pointer. इसे पूरे pointer use करते हैं lexeme के tokens को map करते हैं। Or determine करते हैं।

Syntax Analyzer

Stream
of
tokens



Syntax Analysis/Parsing phase
(Syntax Analyzer, Parser)

→ Parse tree

(CFG)

Parser detect कर Syntax error and semantic error.

Context free Grammar (CFG):

→ Terminal

→ Non terminal

→ Start symbol

production rule : 6

*

$S \rightarrow AC$

$A \rightarrow a | ab$

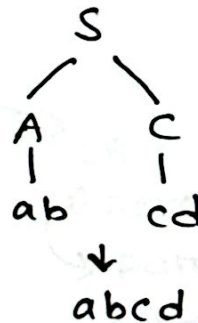
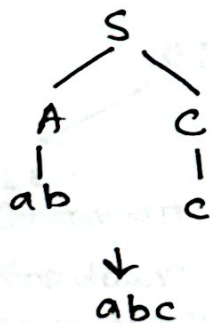
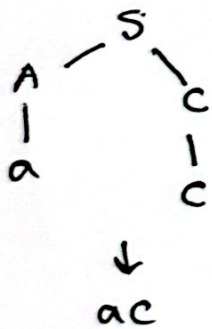
$C \rightarrow c | cd$

non terminal → या production rule

जाह (S, A, C)

terminal → या production rule नह
(a, b, c, d)

start symbol → root node (S)



Parser এর মধ্য যতগুলো rule লিখা থাকে, কোনো rule এর সাথে যদি input না মিলে, তখন syntax error generate হয়।

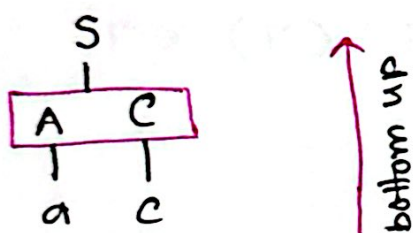
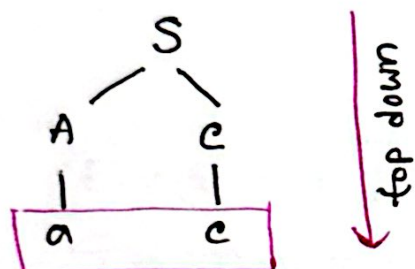
3 types of Parser

↳ top down

↳ bottom up

↳ universal (যেকোনো grammar parse করতে পারে)

input = "ac"

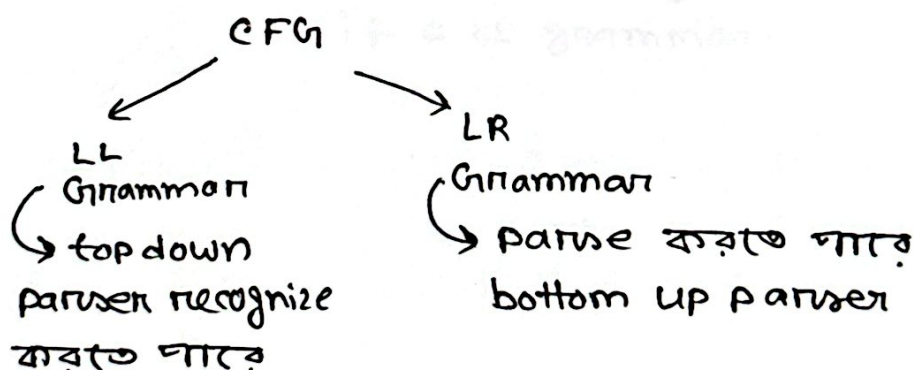


$S \rightarrow AC$

$A \rightarrow a|ab$

$C \rightarrow c|cd$

Terminal symbol থেকে parsing decision carry out করে start symbol এ পৌঁছাতে হয়। যদি পৌঁছাতে পারে, তাহলে সেই particular string এ grammar এ included.



LL Grammar

↳ Leftmost derivation
↳ Left to right

কোনো CFG ত যদি Left recursion যদি থাকে, তাহলে
তা LL Grammar হবে না।

যদি না থাকে, তখন LL Grammar হতে পারে, আবার
নাও হতে পারে।

অর্থাৎ LL Grammar এর necessary condition →

~~Left~~
no Left recursion

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Left recursion exists,

so not a LL Grammar.

Top down parsing not possible.

If we eliminate left recursion
and then check the top down
parser, if the parser can
parse properly, then we call
it a LL Grammar.

Nitrin[®] SR

Nitroglycerin USP 2.6 mg & 6.4 mg Tablet



VESTAR MR[®]

TRIMETAZIDINE

Panic Mode Error Recovery Method

$S \rightarrow T \text{ id};$

$T \rightarrow \text{int} | \text{float}$

* $\text{int } a;$

$\text{int } b$

~~$\text{int } c;$~~

$\text{int } d;$

$\rightarrow$; না পাওয়ায় কারণে panic mode এ ঢল
যাবে এবং পরবর্তী ; পাওয়া পর্যন্ত সব
discard করতে হবে। ; পায়ে ডাল
panic mode ত্যাগ করিয়ে আসবে।

Phrase level recovery

Local correction possible হলে compiler এই correction
করবে।

$\text{int } a;$ $\rightarrow$ missing semicolon add করতে।

Error production add

common error এর জন্য production rule লিখে ফেলা

$S \rightarrow T \text{ id};$

$T \rightarrow \text{int} | \text{float}$

$S \rightarrow T \text{ id}$ $\rightarrow$ ফুল production rule

ফুল production rule use হলে error message দিবে।

Global error recovery $\rightarrow$ costly in time & memory

পূর্বের code এ minimal কতটুকু change করলে correct code
আসবে।

if $(a \geq b)$

$\{$

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet



VESTAR MR[®]
TRIMETAZIDINE

*

$E \rightarrow E + T \mid E - T \mid T$

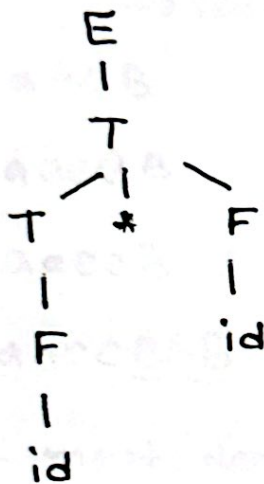
$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow (E) \mid id$

Terminal: +, -, *, /, (,), id

Non-Terminal: E, T, F

Derive $id * id$



$E \Rightarrow T$

$\Rightarrow T * F$

$\Rightarrow F * F$

$\Rightarrow id * F$

$\Rightarrow id * id \rightarrow$ no non-terminal

$E \stackrel{*}{\Rightarrow} id * id$

$E \stackrel{*}{\Rightarrow} \alpha$
 $\hookrightarrow$ sentential form

E থেকে 0 or more steps এ

স্বাক্ষরিত হতে পারে, or sentential form.

sentential form is non-terminal, terminal or both থাকতে পারে,

Left derivation → প্রত্যেকবার left most non-terminal

কো derive করে।

*

$$S \rightarrow aAB$$

$$A \rightarrow AC|ac$$

$$B \rightarrow BAB|b$$

$$c \rightarrow c$$

$$S \xRightarrow{lm} aAB$$

→ left most non-terminal expand

$$\xRightarrow{lm} aACB$$

→ left sentential form

$$\xRightarrow{lm} aacCB$$

$$\xRightarrow{lm} aaccB$$

$$\xRightarrow{lm} aacc\underline{BAB} \quad \text{or} \quad \xRightarrow{lm} aaccb$$

Right most derivation: প্রত্যেকবার right most non-terminal
কো expand করে।

$$S \xRightarrow{rm} aAB$$

$$\xRightarrow{rm} aABAB$$

$$\xRightarrow{rm} aABABb$$

$$\xRightarrow{rm} aABacb$$

$$\xRightarrow{rm} aAbacb$$

$$\xRightarrow{rm} aacbacb$$

→ right sentential form

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet

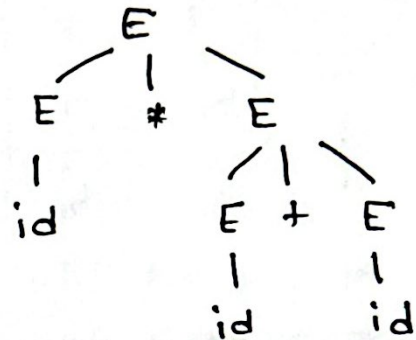
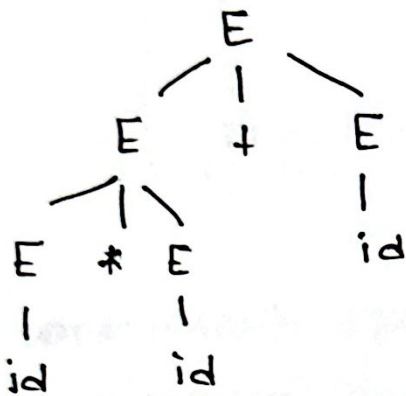


VESTAR MR[®]
TRIMETAZIDINE

જોવાના CFG એક જ અર્થ multiple parse tree possible હોય તો ambiguous.

$E \rightarrow E + E \mid E * E \mid - E \mid (E) \mid id$

derive $id * id + id$



multiple parse tree $\rightarrow$ ambiguous

ambiguous grammar યાચના decision ટિંગાટો નિર્ણય સમુદાય રહે ના.

Bottom up parsing

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$

id * id

$F * id$
 $\mid$
 id

$T * id$
 $\mid$
 F
 $\mid$
 id

$T * F$
 $\mid$
 F
 $\mid$
 id

T
 $\mid$
 $T * F$
 $\mid$
 F
 $\mid$
 id

E
 $\mid$
 T
 $\mid$
 $T * F$
 $\mid$
 F
 $\mid$
 id

পান্নে করে start symbol এ আসতে পারিলে বুঝবে
 bottom up parsing successful.

← nightmost derivation
 $id * id, F * id, T * id, T * F, E$

bottom up parsing এর intermediate step কে reverse দিকে
 consider করেন right most derivation carry out করে।

Handle pruning

(যে তিনিম reduce হচ্ছে, তা handle. প্রথম id reduce
 হয়ে- F হচ্ছে। এখানে প্রথম id হচ্ছে handle.
 Reduce হয়ে non-terminal এ যাওয়াতে prune বানো
 এখানে প্রথম id reduce হয়ে- prune হয়ে F হচ্ছে।

Right sentential form	Handle	Reducing production
$id_1 * id_2$	id_1	$F \rightarrow id$
$F * id_2$	F	$T \rightarrow F$
$T * id_2$	id_2	$F \rightarrow id$
$T * F$	$T * F$	$T \rightarrow T * F$
T	T	$E \rightarrow T$

Shift Reduce Parsing

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

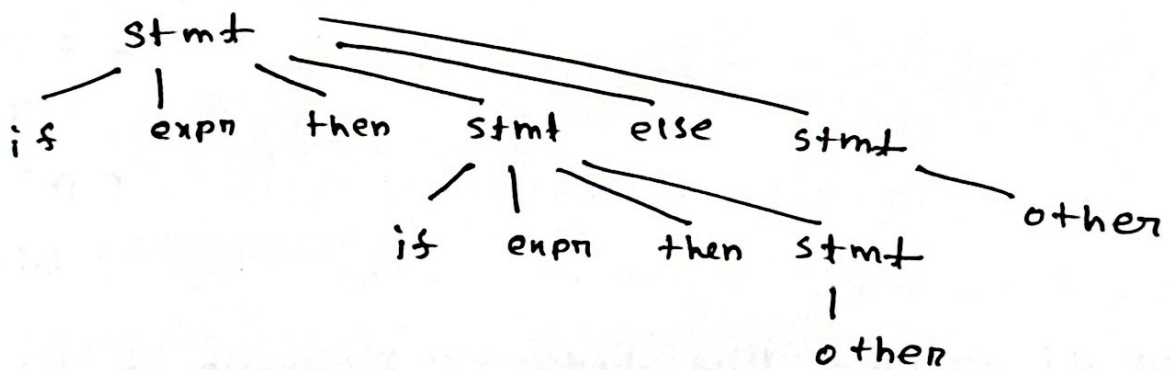
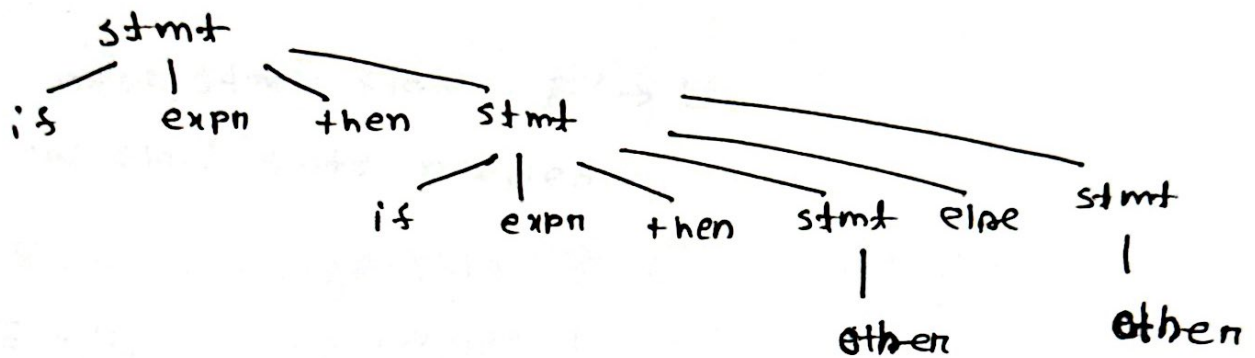
$F \rightarrow (E) \mid id$

Stack	Input	Action
\$	$id_1 * id_2 \$$	shift
\$ id_1	$* id_2 \$$	reduce by $F \rightarrow id$
\$ F	$* id_2 \$$	reduce by $T \rightarrow F$
\$ T	$* id_2 \$$	shift
\$ $T *$	$id_2 \$$	shift
\$ $T * id_2$	$\$$	reduce by $F \rightarrow id$
\$ $T * F$	$\$$	reduce by $T \rightarrow T * F$
\$ T	$\$$	reduce by $E \rightarrow T$
\$ E	$\$$	accept

An Ambiguous grammar can never be LR. তাই (মডি
bottom up way তে parse করতে পারবে না।

stmt $\rightarrow$ if expn then stmt
 $\rightarrow$ if expn then stmt else stmt
 $\rightarrow$ other

* if expn then if expn then other else other



এই string এর জন্য multiple parse tree possible
 হচ্ছে। So grammar ambiguous. তাই shift reduce
 parser parse করতে পারবে না।

LR(0) Automaton

- right most derivation in reverse
- left to right scanning (input)

CFG

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Augment:

new start state: $E' \rightarrow E$

why new start state needed?

$$E' \rightarrow E$$

$$E \rightarrow E + T$$

$$E \rightarrow T$$

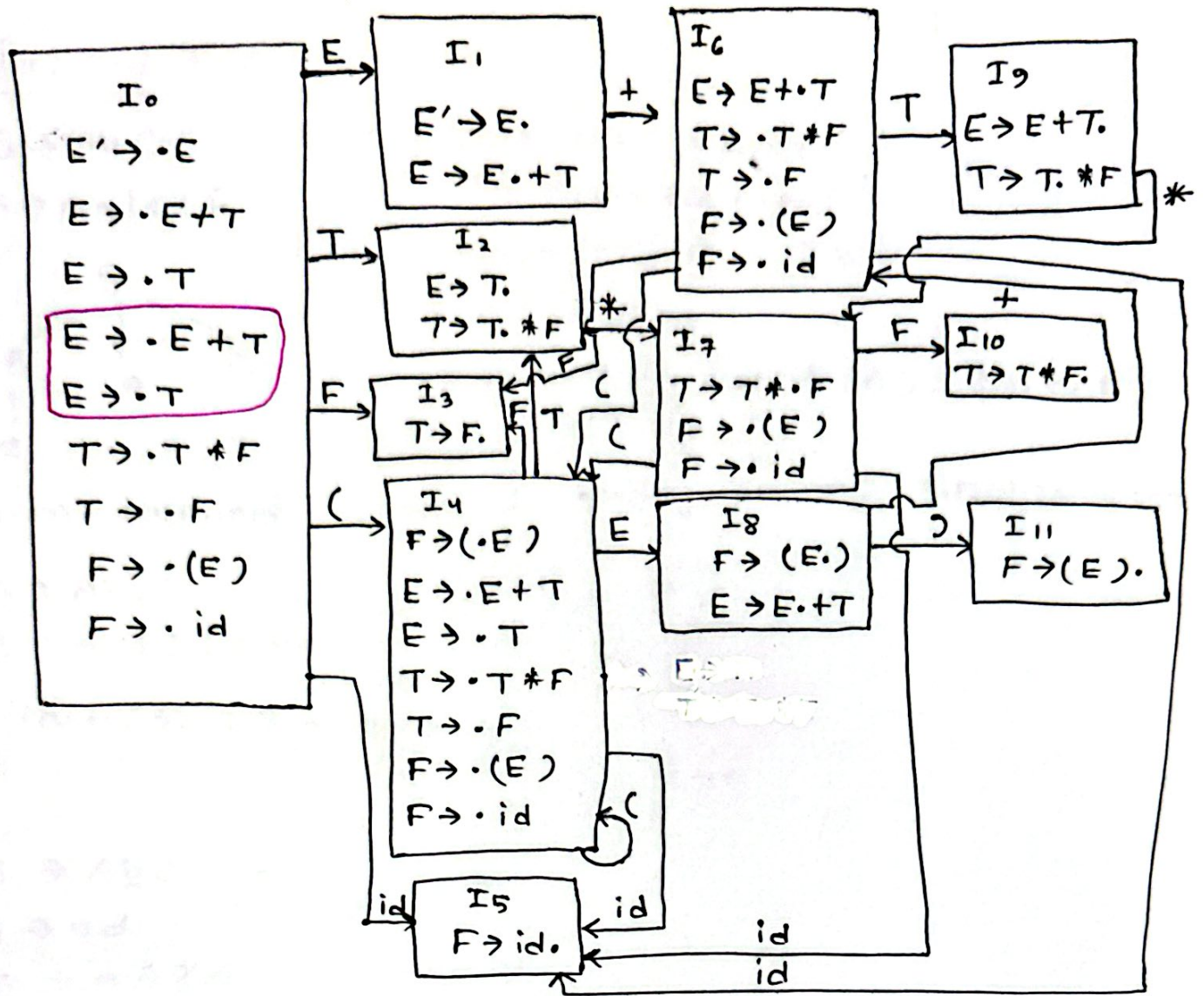
$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow id$$

why we do augment? what will happen if we don't augment?

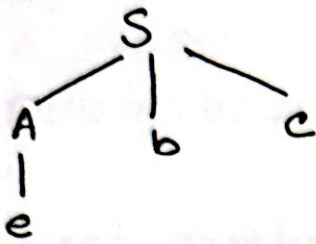


LR(0) input এ রক আছে, তা দেখে না। (যমন: $F \rightarrow (E)$.)
 এখন input এ রক আছে, তা না দেখেই (স reduce করে (যখন)। (যেহেতু input তখন কোনো character পরে না, তাই বলা হয় 0 সফথর lookahead (LR(0)).
 এখন shift reduce conflict arise করে। (যমন I₂.
 $(E \rightarrow T) \rightarrow$ reduce হতে বলে। (অথবা $(T \rightarrow T * F) \rightarrow$ shift হতে বলে
 যা conflict arise করে।
 কোন shift-reduce conflict arise করে ?

First and Follow

$S \rightarrow Abc$

$A \rightarrow a|e|f$



$\text{First}(A) = \{a, e, f\}$

↳ non-terminal

$\text{First}("bc") = \{b\}$

↳ string

$\text{First}(S) = \text{First}(A) = \{a, e, f\}$

non-terminal, terminal, string એ ત્રણ First નિર્ધારિત છે.

$S \rightarrow Abc$

$A \rightarrow a|e|f| \epsilon$

$\text{First}(S) = \{a, e, f, b\}$

↳ $S \rightarrow \epsilon bc$

$S \rightarrow A \underline{b} c$

$A \rightarrow a d$

$P \rightarrow a A \underline{x} d$

$\text{Follow}(A) = \{b, x\}$

non-terminal એ ત્રણ follow નો વર્ણન કરે છે.

$A \rightarrow \alpha B \gamma$

$\text{Follow}(B) \rightarrow \text{First}(\gamma)$ (જો γ નો શરૂઆતનો ચારે છે.)

$S \rightarrow \underline{\epsilon} \underline{A} \underline{b} \underline{c}$
 $\quad \alpha \quad B \quad \gamma$

$\text{Follow}(A) = \text{First}(bc) = \{b\}$

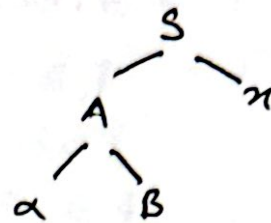
$$A \rightarrow \alpha B$$

Follow (B) will include Follow (A)

$$S \rightarrow A \alpha$$

$$A \rightarrow \alpha B$$

$$\text{Follow}(B) = \{ \alpha \}$$



Start symbol is Follow to \$ symbol

*

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

First calculation

$$\begin{array}{l} \uparrow \\ E \rightarrow \{(, id\} \\ T \rightarrow \{(, id\} \\ E' \rightarrow \{+, \epsilon\} \\ F \rightarrow \{(, id\} \\ T' \rightarrow \{*, \epsilon\} \end{array}$$

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet



VESTAR MR[®]
TRIMETAZIDINE

*

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT' | \epsilon$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

First

$$\begin{aligned} E &\rightarrow \{ (, id, +, E' \} \\ T &\rightarrow \{ (, id, \epsilon \} \\ E' &\rightarrow \{ +, \epsilon \} \\ F &\rightarrow \{ (, id \} \\ T' &\rightarrow \{ *, \epsilon \} \end{aligned}$$

$$First(TE') = First(T) = First(E')$$

$$if T = \epsilon$$

$$then E \rightarrow \epsilon E'$$

$$in that case First(E) = First(E') = \{ +, \epsilon \}$$

Follow calculation

$$\begin{aligned} E &\rightarrow \{), \$ \} \rightarrow A \rightarrow \alpha B \downarrow \\ T &\rightarrow \{ +,), \$ \} \rightarrow A \rightarrow \alpha B \quad (E \rightarrow TE) \\ &\quad \text{follow}(T) \rightarrow \text{follow}(E) \\ E' &\rightarrow \{), \$ \} \rightarrow A \rightarrow \alpha B \\ F &\rightarrow \{ *, +,), \$ \} \\ T' &\rightarrow \{ +,), \$ \} \end{aligned}$$

$$E' \rightarrow \{ +, \epsilon \}$$

$$\hookrightarrow E \rightarrow \alpha B$$

$$\hookrightarrow E \rightarrow \alpha B$$

$$so First(E) \rightarrow \{), \$ \}$$

$$\hookrightarrow follow(T) \rightarrow follow(E)$$

Follow to ϵ বসে না।

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet



VESTAR MR[®]
TRIMETAZIDINE

* $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$

First

$E \rightarrow \{ (, id \}$
 $T \rightarrow \{ (, id \}$
 $F \rightarrow \{ (, id \}$

Follow

$E \rightarrow \{), \$, + \}$
 $T \rightarrow \{), \$, +, * \}$
 $F \rightarrow \{), \$, +, * \}$

Follow ଟ- \$ ଆକେ କେନ ? ϵ ଆକେ ନା କେନ ?

- କିନ୍ତୁ ନା ଆବଶ୍ୟକ ଓ \$ ଆକେ । ତାହା follow ଟେ \$
ଆକେ, ϵ ଆକେ ନା ।

SLR(1) $\rightarrow$ simple LR parser

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet


Healthcare

VESTAR MR[®]
TRIMETAZIDINE

LR Parsing table

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Augment

$E' \rightarrow E$

1. $E \rightarrow E + T$

2. $E \rightarrow T$

3. $T \rightarrow T * F$

4. $T \rightarrow F$

5. $F \rightarrow (E)$

6. $F \rightarrow id$

State	Action						Go-to		
	+	*	(	)	id	\$	E	T	F
0			S4		S5		1	2	3
1	S6					accept			
2	R2	S7/ R2	R2	R2	R2	R2			
3	R4	R4	R4	R4	R4	R4			
4			S4		S5		8	2	3
5	R6	R6	R6	R6	R6	R6			
6			S4		S5			9	3
7			S4		S5				10
8	S6			S11					
9	R1	S7/ R1	R1	R1	R1	R1			
10	R3	R3	R3	R3	R3	R3			
11	R5	R5	R5	R5	R5	R5			

SLR Panner table

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow id$

$$\text{First}(E) = \{ (, id \}$$

$$\text{First}(T) = \{ (, id \}$$

$$\text{First}(F) = \{ (, id \}$$

$$\text{Follow}(E) = \{ \$, +,) \}$$

$$\text{Follow}(T) = \{ *, \$, +,) \}$$

$$\text{Follow}(F) = \{ *, \$, +,) \}$$

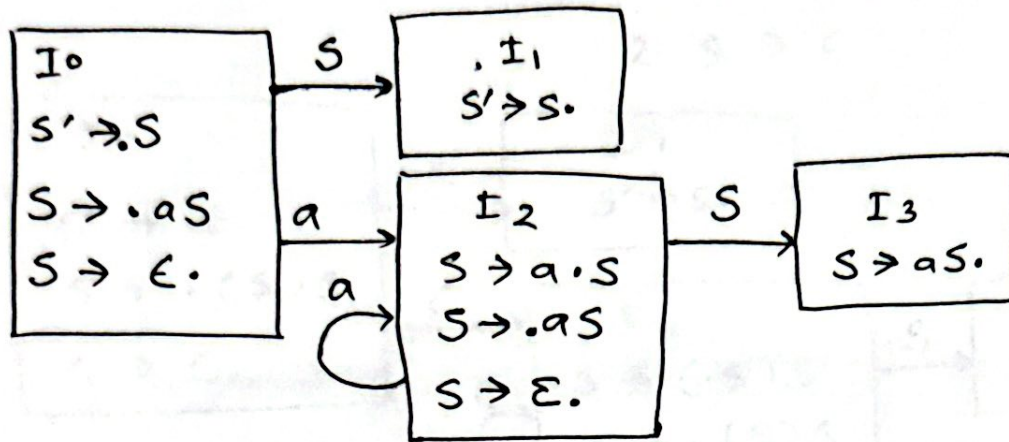
Serial	Action						Go-to		
	id	+	*	(	)	\$	E	T	F
0	S5			S4			1	2	3
1		S6				Accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

LR(0) Automaton construction with Epsilon Rules

$S \rightarrow aS \mid \epsilon$

Augment:

$S' \rightarrow S$
 $S \rightarrow aS \mid \epsilon \rightarrow 1. S \rightarrow aS$
 $2. S \rightarrow \epsilon$



$\text{Follow}(S) = \{\$, \epsilon\}$

SLR(1) Parse table

State	Action			Go to
	a	ϵ	$\$$	
0	SR		R2	1
1			accept	
2	SR		R2	3
3			R1	

Example 2: LR(0) Automaton with Epsilon

$S \rightarrow (S)S$

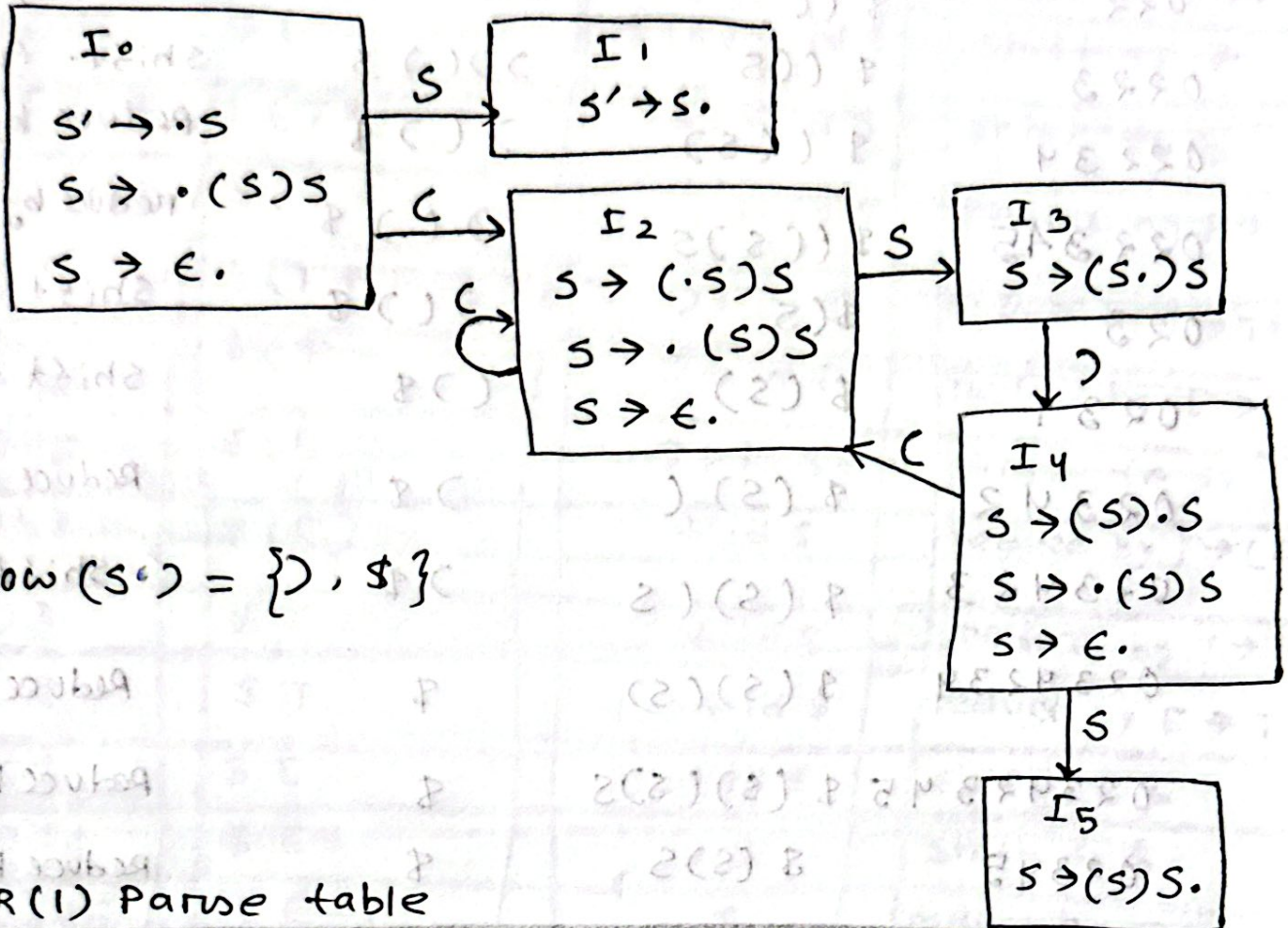
$S \rightarrow \epsilon$

Augment:

$S' \rightarrow S$

1. $S \rightarrow (S)S$

2. $S \rightarrow \epsilon$



Follow(S') = $\{), \$\}$

SLR(1) Parse table

state	Action				Go to
	C	)	ε	\$	
0	S2	R2		R2	1
1				Accept	
2	S3	R2		R2	3
3		S4			
4	S2	R2		R2	5
5		R1		R1	

SLR Parsing Algorithm simulation

Stack	Symbol	Input	Action
0	\$	(()) () \$	Shift to 2
0 2	\$ (	() () \$	Shift to 2
0 2 2	\$ ((	)) () \$	Reduce by $S \rightarrow \epsilon$
0 2 2 3	\$ ((S	)) () \$	Shift to 4
0 2 2 3 4	\$ ((S)	) () \$	Reduce by $S \rightarrow \epsilon$
0 2 2 3 4 5	\$ ((S)S	) () \$	Reduce by $S \rightarrow (S)S$
0 2 3	\$ (S	) () \$	Shift to 4
0 2 3 4	\$ (S)	() \$	Shift to 2
0 2 3 4 2	\$ (S) (	) \$	Reduce by $S \rightarrow \epsilon$
0 2 3 4 2 3	\$ (S) (S	) \$	Shift to 4
0 2 3 4 2 3 4	\$ (S) (S)	\$	Reduce by $S \rightarrow \epsilon$
0 2 3 4 2 3 4 5	\$ (S) (S)S	\$	Reduce by $S \rightarrow (S)S$
0 2 3 4 5	\$ (S)S	\$	Reduce by $S \rightarrow (S)S$
0 1	\$ S	\$	Accept

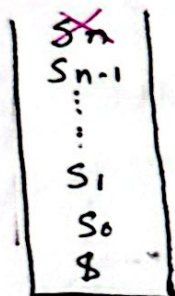
SLR Parsing Algorithm Simulation

Stack	Symbol	Input	Action
0	\$	(id * id) + id \$	Shift to 4
04	\$ (	id * id) + id \$	Shift to 5
045	\$ (id	* id) + id \$	reduce by $F \rightarrow id$
043	\$ (F	* id) + id \$	reduce by $T \rightarrow F$
042	\$ (T	* id) + id \$	Shift to 7
0427	\$ (T*	id) + id \$	Shift to 5
04275	\$ (T*id	) + id \$	reduce by $F \rightarrow id$
042710 pop	\$ (T*F	) + id \$	reduce by $T \rightarrow T*F$
042 push	\$ (T	) + id \$	reduce by $E \rightarrow T$
048	\$ (E	) + id \$	Shift to 11
04811	\$ (E)	+ id \$	reduce by $F \rightarrow (E)$
03	\$ F	+ id \$	reduce by $T \rightarrow F$
02	\$ T	+ id \$	reduce by $E \rightarrow T$
01	\$ E	+ id \$	Shift to 6
016	\$ E+	id \$	Shift to 5
0165	\$ E+id	\$	reduce by $F \rightarrow id$
0163	\$ E+F	\$	Reduce by $T \rightarrow F$
0169	\$ E+T	\$	reduce by $E \rightarrow E+T$
01	\$ E	\$	Accept

Parusing simulation of a string that causes syntax error

Stack	Symbol	input	Action
0	\$	id +) \$	shift to 5
05	\$ id	+) \$	reduce by $F \rightarrow id$
03	\$ F	+) \$	reduce by $T \rightarrow F$
02	\$ T	+) \$	reduce by $E \rightarrow T$
01	\$ E	+) \$	shift to 6
016	\$ E +	) \$	Syntax error

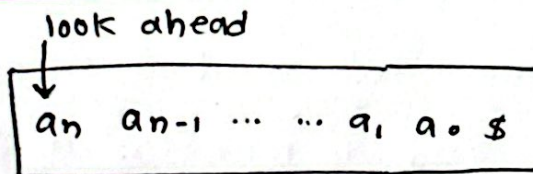
Panic Mode Error Recovery



state stack

Part discard

S_n and a_n are corresponding action possible
 ના રાશિ S_n ને discard કરાવો. Then S_{n-1} and
 a_n એ અનુ-રૂપ action વાજે, જા valid. એવા
 parsing continue રાજ.



Future discard

S_n
S_{n-1}
$\vdots$
$\vdots$
S_0
$\$$

$\downarrow$
 ~~a_n~~ a_{n-1} ... a_0 $\$$

S_n and a_n are corresponding action at present
 a_n discard karat. Then S_n and a_{n-1} are
 corresponding action check karat.

Input taraka symbol discard $\rightarrow$ Future discard
 state stack taraka discard $\rightarrow$ past discard



a_n
a_{n-1}
$\vdots$
$\vdots$
a_0
$\$$

Phase Level Error Recovery

int a;

int b;

প্রথম লাইনে ; না থাকলে phase level error
recovery-র জন্য ব্যবহৃত হয়, এবং parsing carry
out করা হয়। একে local correction বলা হয়।

একটি বাক্যের মধ্যে একটি বাক্যের মতো
একটি বাক্যের মধ্যে একটি বাক্যের মতো
একটি বাক্যের মধ্যে একটি বাক্যের মতো
একটি বাক্যের মধ্যে একটি বাক্যের মতো
একটি বাক্যের মধ্যে একটি বাক্যের মতো
একটি বাক্যের মধ্যে একটি বাক্যের মতো
একটি বাক্যের মধ্যে একটি বাক্যের মতো
একটি বাক্যের মধ্যে একটি বাক্যের মতো
একটি বাক্যের মধ্যে একটি বাক্যের মতো
একটি বাক্যের মধ্যে একটি বাক্যের মতো

The operation that need to be performed on
the result and a decision to be taken
initially a decision.

1. scope error
2. scope error
3. scope error
4. scope error
5. scope error
6. scope error
7. scope error
8. scope error
9. scope error
10. scope error

Symbol Table

Identifier and token and information regarding.

What identifier and info regarding?

- semantic check and - identifier initialize
regarding that, what and datatype which is function
which is specific value return regarding that, return type which is function regarding parameter
which is, regarding parameter and datatype which is, or an array and index and value regarding, and array and index and length regarding that, and information regarding identifier and its search regarding that, and token and its regarding that.

The operations that need to be performed on scope entry/exit, and to process a declaration/use are:

- i. scope entry
- ii. process a declaration
- iii. process a use
- iv. scope exit

can be implemented through one of two methods:

- i. List of hash tables
- ii. hash tables of list.

List of hashtable

Basically प्रत्येक scope এর জন্য আলাদা আলাদা table থাকে।

1. int a;
2. void func(int i) {
3. i = i + 1;
4. if (i > 0) {
5. int f;
6. }
7. }
8. char c;

normal → data type

function → return type

num of param

table length = 3

hash function

a → 1

a: int

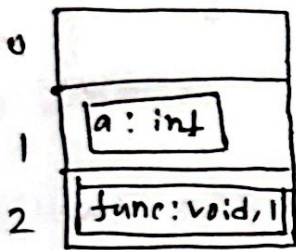
func → 2

func: void, 1

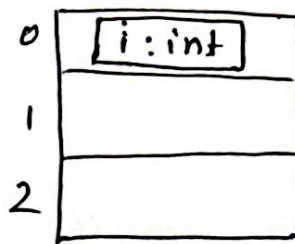
i → 0

i: int

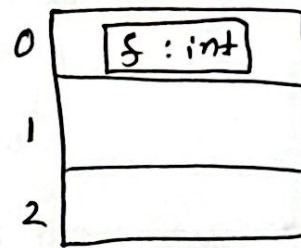
level = 0, 1, 2, 3



(1)



(2)

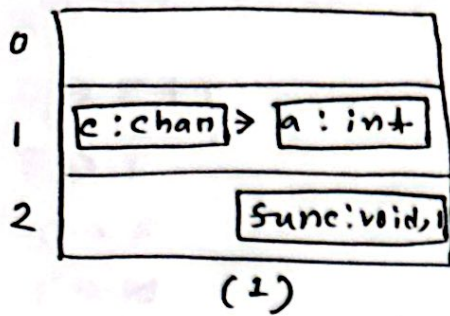


(3)

head

hash function দিয়ে index generate করতে হবে। 0 no node generate করলে search করতে হবে (কোন node exist করে কিনা, যদি exist ~~no time~~ করে then tell 'multiple declared variable', exist না করলে insert করে দিবে।

6 no line এ scope exit হয়েছ। scope exit হলে level 20 value 1 minus করতে হবে এবং list এর current head কে remove করতে হবে।



$c \rightarrow 1$ c:chan

1 no node is already exist
 કરતો, so collision રહતો, આ
 solve કરતો forward chaining
 use કરતો. કારણે head નો
 નવું નોડ add કરતો.

Hash tables of Link

1. int a;
2. void func(int i) {
3. i = i + 1;
4. if (i > 0) {
5. int f;
6. }
7. }
8. char c;

array $\rightarrow$ datatype, level, length
 normal $\rightarrow$ datatype, level num
 function $\rightarrow$ return type, num
 of param, level num

total length = 5

hash index

a $\rightarrow 0$

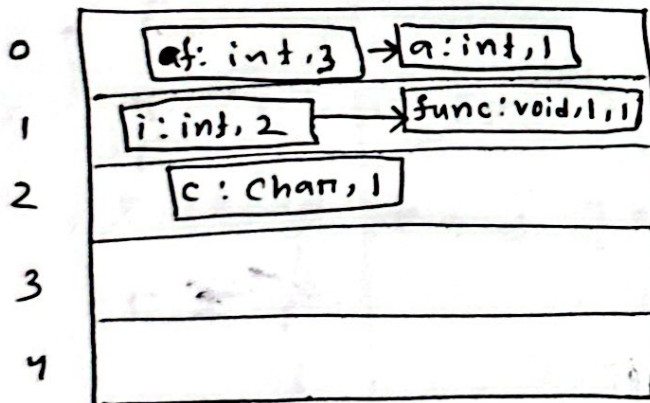
c $\rightarrow 2$

func $\rightarrow 1$

i $\rightarrow 1$

f $\rightarrow 0$

level = 0, 1, 2, 3



no line નો scope exit. scope
 exit રહેલો search કરતો રહતો (પરંતુ current
 level નો તેજા નોડ આપેલો કિના,
 આજનાં કિના. (એક level 3 નો f દેખે.
 0-4 search કરતો every level નો જાય.)

i નો level number 2,
 યાદ માટે નોડ 1 નો
 already existed func નો
 level value compare કરતો.
 match નો રહેલો comparison
 નો કિયુ નેઈ.

f નો એક આજ 0 નો
 node index નો એક આજ
 node exist કરતો કિના આ
 key નો f આજ, આ search
 કરતો રહતો. key exist કરતો
 (multiple declared variable
 નો insert કરતો.)

Nitrin[®] SR

Nitroglycerin USP 2.6 mg & 6.4 mg Tablet



Healthcare

VESTAR MR[®]

TRIMETAZIDINE

Lecture 9: Adding Semantic meaning for a CFG

$E \rightarrow E + T$

input string: 2+3

$E \rightarrow T$

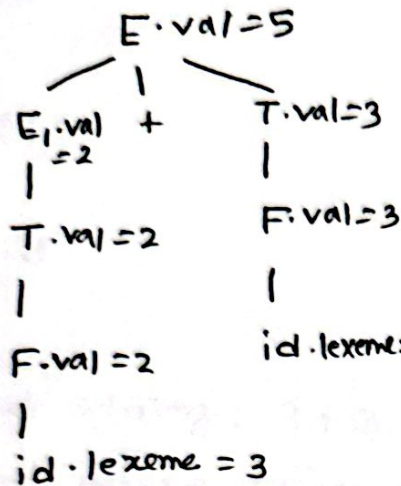
post order: visit n. children visit,
then visit(n).

$T \rightarrow F$

$F \rightarrow id$

pre order: visit visit(n),
then n. children visit

Annotated parse tree:



In order: visit left children
visit, then n visit
and then right children visit

We use post order traversal here.

Stack:

id
|
F
|
T
|
E + id
|
E + F
|
E + T

$E \rightarrow E + T \quad \{ E.val = E1.val + T.val \}$

$E \rightarrow T \quad \{ E.val = T.val \}$

$T \rightarrow F \quad \{ T.val = F.val \}$

$F \rightarrow id \quad \{ F.val = id.lexeme \}$

*

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

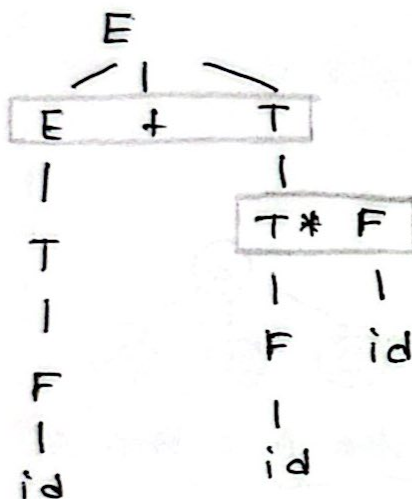
$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

Input string: $2 + 3 * 5$

precedence বন্ধি হলে তা parse tree র নিচে রাখতে,
যে অংশ execute হয়।



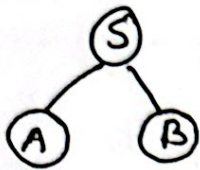
এখানে precedence $T * F$ parse tree এর নিচে।

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet



VESTAR MR[®]
TRIMETAZIDINE

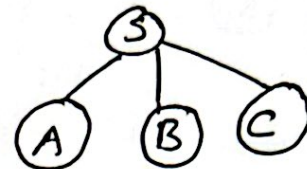
Synthesized attribute: child ka nikal kr upar dependent



$$S \rightarrow AB \quad \{ S.attr = A.attr * B.attr \}$$

Inherited attribute: parent / sibling / self attribute

$$\{ A.attr = S.attr + C.attr \}$$

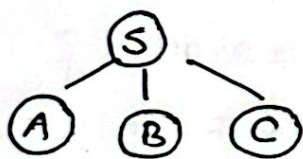


attributed grammar: syntactic rule + CFG

S attributed grammar: synthesized

L attributed grammar: synthesized / inherited

↳ only take
 ↳ left sibling value
 ↳ parent
 ↳ self



$$\{ B.attr = \underbrace{A.attr}_{\text{left sibling}} + \underbrace{C.attr}_{\text{right sibling}} \} \rightarrow \text{inherited not L attributed}$$

$$\{ B.attr = A.attr \} \rightarrow \text{inherited and L attributed}$$

$$\{ B.attr = \underbrace{A.attr}_{\text{left sibling}} + \underbrace{S.attr}_{\text{parent}} \} \rightarrow \text{inherited and L attributed}$$

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet

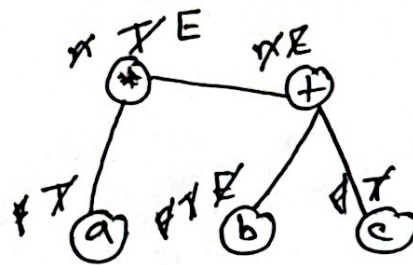
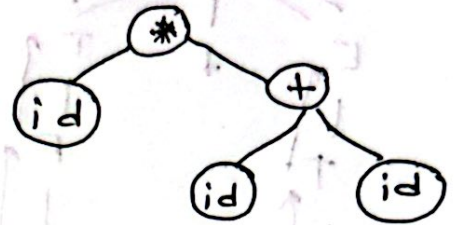
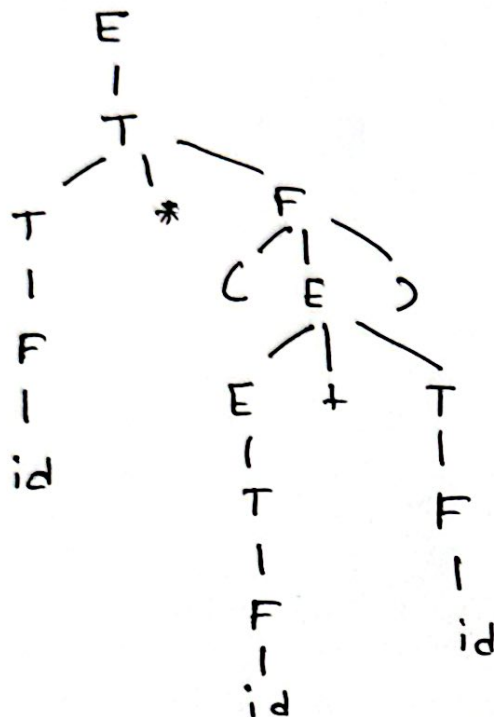


VESTAR MR[®]
TRIMETAZIDINE

Lecture 10: Abstract syntax tree

Abstract syntax tree is a compact version of parse tree.

input string: $a * (b + c) \rightarrow id * (id + id)$



$$E \rightarrow E + T \quad \left\{ \begin{array}{l} n = \text{new Node}('+'); \\ n.\text{child}.\text{append}(E, \text{node}); \\ n.\text{child}.\text{append}(T.\text{node}); \\ E.\text{node} = n; \end{array} \right.$$

$$E \rightarrow T \quad \{ E.\text{node} = T.\text{node} \}$$

$$T \rightarrow T * F \quad \left\{ \begin{array}{l} n = \text{new node}('*'); \\ n.\text{child}.\text{append}(T.\text{node}); \\ n.\text{child}.\text{append}(F.\text{node}); \\ T.\text{node} = n; \end{array} \right.$$

$$T \rightarrow F \quad \{ T.\text{node} = F.\text{node} \}$$

$$F \rightarrow (E) \quad \{ F.\text{node} = E.\text{node} \}$$

$$F \rightarrow id \quad \{ F.\text{node} = \text{new leaf}(id.\text{lexeme}); \}$$

Dependency graph

input string: id + id

